

Controllo del flusso

break e continue

Come controllare il flusso dei cicli con break e continue in C++.

Controllare il flusso dentro un ciclo

A volte, mentre stai scorrendo una lista, vuoi fermarti non appena hai trovato quello che cercavi. Oppure vuoi saltare certi elementi e andare direttamente al prossimo.

`break` e `continue` sono due istruzioni che ti danno questo controllo preciso sul comportamento dei cicli.

break : esci dal ciclo

`break` interrompe immediatamente il ciclo. L'esecuzione continua con la prima istruzione dopo il ciclo, come se il ciclo fosse finito.

```
for (int i = 0; i < 10; i++) {  
    if (i == 5) {  
        break; // usciamo quando i vale 5  
    }  
    cout << i << " ";  
}  
cout << "fine";  
// Output: 0 1 2 3 4 fine
```

Quando usare break

Il caso più comune: cercare qualcosa in una lista. Appena lo trovi, non ha senso continuare a cercare:

```

int numeri[] = {10, 25, 3, 47, 8, 15};
int dimensione = 6;
int daQuerare = 47;
bool trovato = false;

for (int i = 0; i < dimensione; i++) {
    if (numeri[i] == daQuerare) {
        cout << "Trovato " << daQuerare << " alla posizione " << i << endl;
        trovato = true;
        break; // trovato! inutile continuare a cercare
    }
}

if (!trovato) {
    cout << daQuerare << " non trovato." << endl;
}

```

break nei cicli annidati

break esce solo dal ciclo più interno in cui si trova. Il ciclo esterno continua normalmente:

```

for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        if (j == 1) {
            break; // esce solo dal ciclo interno (j)
        }
        cout << "(" << i << "," << j << ") ";
    }
    cout << endl;
}

```

Output:

```

(0,0)
(1,0)
(2,0)

```

continue : salta al prossimo giro

`continue` non esce dal ciclo. Salta solo il resto del blocco corrente e passa direttamente alla prossima iterazione.

```
for (int i = 0; i < 10; i++) {  
    if (i % 2 == 0) {  
        continue; // se il numero è pari, saltiamo questa iterazione  
    }  
    cout << i << " "; // stampiamo solo i dispari  
}  
// Output: 1 3 5 7 9
```

Quando usare `continue`

Utile per saltare certi casi senza dover annidare tutto il codice dentro un `if` :

```
// Stampa solo i numeri non divisibili per 3  
for (int i = 1; i <= 20; i++) {  
    if (i % 3 == 0) {  
        continue; // saltiamo i multipli di 3  
    }  
    cout << i << " ";  
}  
// Output: 1 2 4 5 7 8 10 11 13 14 16 17 19 20
```

Un altro esempio: ignorare i dati non validi e continuare con quelli validi:

```

int voti[] = {8, -1, 7, 150, 6, 9};
int n = 6;
int somma = 0;
int contatore = 0;

for (int i = 0; i < n; i++) {
    if (voti[i] < 0 || voti[i] > 10) {
        cout << "Voto non valido ignorato: " << voti[i] << endl;
        continue; // saltiamo questo voto e passiamo al prossimo
    }
    somma += voti[i];
    contatore++;
}

if (contatore > 0) {
    cout << "Media voti validi: " << (double)somma / contatore << endl;
}

```

break e continue nel while

Funzionano allo stesso modo anche in `while` e `do...while` :

```

// Ciclo "infinito" controllato da break
int i = 0;
while (true) {
    if (i >= 5) {
        break; // uscita manuale dal ciclo
    }
    cout << i << " ";
    i++;
}
// Output: 0 1 2 3 4

```

```
int i = 0;
while (i < 10) {
    i++;
    if (i == 5) {
        continue; // salta il 5
    }
    cout << i << " ";
}
// Output: 1 2 3 4 6 7 8 9 10
```

Consigli pratici

- Usa `break` e `continue` con moderazione. Troppi rendono il codice difficile da seguire.
- Se un ciclo ha molti `break`, potrebbe essere riscrivibile con una condizione più precisa:

```
// Con break (meno chiaro)
for (int i = 0; i < 100; i++) {
    if (i * i > 50) {
        break;
    }
    cout << i << " ";
}

// Alternativa più chiara: la condizione di uscita è nella testa del for
for (int i = 0; i < 100 && i * i <= 50; i++) {
    cout << i << " ";
}
```

- Se usi `break` o `continue` per un motivo non ovvio, aggiungi un commento che spiega il perché.

Condizioni in C++

Prendere decisioni nel codice con `if`, `else if`, `else` e `switch` in C++.

Perché le condizioni?

Un programma che fa sempre la stessa cosa non è molto utile. I programmi interessanti prendono **decisioni**: se l'utente è maggiorenne, mostra il contenuto; se il voto è sufficiente, passa alla classe successiva; se piove, porta l'ombrello.

Le condizioni permettono al programma di scegliere tra strade diverse in base a una situazione.

L'istruzione **if**

if significa "se". Se la condizione tra parentesi è vera, il blocco di codice viene eseguito. Altrimenti viene saltato.

```
if (condizione) {  
    // questo codice viene eseguito solo se la condizione è vera  
}
```

```
int eta = 18;  
  
if (eta >= 18) {  
    cout << "Sei maggiorenne." << endl;  
}  
// Se eta fosse 15, questa riga non verrebbe mai stampata
```

Nota: in C++ la condizione **deve** stare tra parentesi tonde **()**.

L'istruzione **else**

else significa "altrimenti". Definisce cosa fare quando la condizione è falsa:

```
int eta = 15;

if (eta >= 18) {
    cout << "Sei maggiorenne." << endl;
} else {
    cout << "Sei minorenni." << endl;
}

// Stampa: "Sei minorenni."
```

L'istruzione **else if**

Quando hai più di due possibilità, usa **else if** per controllare più condizioni in sequenza:

```
int voto = 75;

if (voto >= 90) {
    cout << "Ottimo!" << endl;
} else if (voto >= 70) {
    cout << "Buono!" << endl;      // ← questa viene stampata
} else if (voto >= 60) {
    cout << "Sufficiente." << endl;
} else {
    cout << "Insufficiente." << endl;
}
```

Il programma controlla le condizioni dall'alto verso il basso e si ferma alla prima che risulta vera. Le successive vengono ignorate.

Condizioni annidate

Puoi mettere un **if** dentro un altro **if**. Pensa a esso come a una serie di domande: prima la domanda principale, poi le sotto-domande.

```

int eta = 20;
bool haPatente = true;

if (eta >= 18) {
    // siamo maggiorenni – ora controlliamo la patente
    if (haPatente) {
        cout << "Puoi guidare." << endl;
    } else {
        cout << "Sei maggiorenne ma non hai la patente." << endl;
    }
} else {
    cout << "Sei minorenne, non puoi guidare." << endl;
}

```

Usare gli operatori logici

Puoi combinare più condizioni in una sola usando `&&` (E), `||` (O) e `!` (NON):

```

int eta = 16;
bool conAccompagnatore = true;

// Può entrare se è maggiorenne OPPURE se ha un accompagnatore
if (eta >= 18 || conAccompagnatore) {
    cout << "Puoi entrare." << endl;
}

```

```

int temperatura = 22;
bool piove = false;

// Perfetto per uscire se è caldo E non piove
if (temperatura > 20 && !piove) {
    cout << "Perfetto per uscire!" << endl;
}

```


Il costrutto `switch`

Quando devi confrontare una variabile con molti valori fissi, `switch` è più leggibile di una lunga serie di `else if`. Funziona come un selettore: vai direttamente al caso corrispondente.

```
int giorno = 3;

switch (giorno) {
    case 1:
        cout << "Lunedì" << endl;
        break;
    case 2:
        cout << "Martedì" << endl;
        break;
    case 3:
        cout << "Mercoledì" << endl; // ← questo viene eseguito
        break;
    case 4:
        cout << "Giovedì" << endl;
        break;
    case 5:
        cout << "Venerdì" << endl;
        break;
    default:
        cout << "Weekend" << endl; // eseguito se nessun case
        corrisponde
}
```

Output: Mercoledì

I componenti di `switch`

- **case valore:** — esegue il codice se la variabile vale quel numero
- **break** — esce dallo switch. Senza di esso, l'esecuzione continua nei case successivi!
- **default** — viene eseguito se nessun case corrisponde (facoltativo)

Attenzione al `break` !

Dimenticare il `break` è un errore comune. Senza di esso, dopo aver trovato il case giusto il programma continua ad eseguire anche i case successivi:

```

int x = 2;
switch (x) {
    case 1:
        cout << "uno" << endl;
    case 2:
        cout << "due" << endl;    // eseguito (corretto)
    case 3:
        cout << "tre" << endl;    // eseguito anche questo! (manca break)
        break;
}
// Output: due
//         tre

```

A volte questo comportamento è intenzionale. Per esempio, se vuoi che più case eseguano lo stesso codice:

```

char voto = 'B';
switch (voto) {
    case 'A':
    case 'B':
        // sia A che B arrivano qui
        cout << "Promosso con lode" << endl;
        break;
    case 'C':
        cout << "Promosso" << endl;
        break;
    default:
        cout << "Bocciato" << endl;
}

```

L'operatore ternario

Per condizioni semplici con solo due risultati, esiste una forma compatta su una riga sola:

```

// condizione ? valore_se_vero : valore_se_falso
int eta = 20;
string stato = (eta >= 18) ? "maggiorenne" : "minorenne";
cout << stato << endl;    // maggiorenne

```

Usa questa forma solo quando è chiara da leggere. Se la condizione è complessa, preferisci un normale `if / else`.

Esempio pratico

```
#include <iostream>
using namespace std;

int main() {
    int punteggio;
    cout << "Inserisci il punteggio (0-100): ";
    cin >> punteggio;

    // Prima controlliamo che il valore sia valido
    if (punteggio < 0 || punteggio > 100) {
        cout << "Punteggio non valido!" << endl;
    } else if (punteggio >= 90) {
        cout << "Voto: A – Eccellente!" << endl;
    } else if (punteggio >= 80) {
        cout << "Voto: B – Molto buono" << endl;
    } else if (punteggio >= 70) {
        cout << "Voto: C – Buono" << endl;
    } else if (punteggio >= 60) {
        cout << "Voto: D – Sufficiente" << endl;
    } else {
        cout << "Voto: F – Insufficiente" << endl;
    }

    return 0;
}
```

Ciclo do...while

Il ciclo do...while in C++, che esegue il blocco almeno una volta.

do-while : quando eseguire almeno una volta

A volte vuoi che il tuo programma faccia qualcosa almeno una volta, e poi decida se ripeterlo.

Per esempio: mostra il menu all'utente, poi chiedi se vuole continuare. Il menu deve apparire sempre almeno una volta, anche prima di sapere cosa vuole l'utente.

Il ciclo `do...while` fa esattamente questo: **esegue il blocco, poi controlla la condizione**.

La struttura del `do...while`

```
do {  
    // codice da eseguire almeno una volta  
} while (condizione);
```

Nota il **punto e virgola** dopo la parentesi tonda finale — è obbligatorio e facile da dimenticare.

Confronto con il `while` normale

Il `while` controlla la condizione **prima** di eseguire il blocco. Se la condizione è già falsa, non esegue mai nulla:

```
int x = 10;  
while (x < 5) {  
    cout << "while: " << x << endl; // non viene mai stampato  
}
```

Il `do...while` esegue il blocco **almeno una volta**, poi controlla:

```
int y = 10;  
do {  
    cout << "do-while: " << y << endl; // viene stampato una volta  
} while (y < 5);
```

Output:

```
do-while: 10
```

Anche se `y = 10` non è minore di 5, il blocco viene eseguito lo stesso la prima volta.

Il caso d'uso classico: validare l'input

Il `do...while` è perfetto quando vuoi chiedere qualcosa all'utente e ripetere finché non inserisce un valore corretto:

```
int numero;

do {
    cout << "Inserisci un numero tra 1 e 10: ";
    cin >> numero;

    if (numero < 1 || numero > 10) {
        cout << "Valore non valido, riprova." << endl;
    }
} while (numero < 1 || numero > 10);

cout << "Hai inserito: " << numero << endl;
```

Con il `while` normale, dovresti leggere l'input anche prima del ciclo (duplicando il codice). Con `do...while` il codice è più pulito.

Menu interattivo

Il `do...while` è ideale per i menu: il menu deve sempre apparire almeno una volta, e si ripete finché l'utente non sceglie di uscire.

```

#include <iostream>
using namespace std;

int main() {
    int scelta;

    do {
        // Il menu viene mostrato all'inizio di ogni giro
        cout << "\n=== Menu ===" << endl;
        cout << "1. Saluta" << endl;
        cout << "2. Conta fino a 5" << endl;
        cout << "3. Esci" << endl;
        cout << "Scelta: ";
        cin >> scelta;

        switch (scelta) {
            case 1:
                cout << "Ciao! Come stai?" << endl;
                break;
            case 2:
                for (int i = 1; i <= 5; i++) {
                    cout << i << " ";
                }
                cout << endl;
                break;
            case 3:
                cout << "Arrivederci!" << endl;
                break;
            default:
                cout << "Scelta non valida." << endl;
        }

    } while (scelta != 3); // ripete finché l'utente non sceglie "Esci"

    return 0;
}

```

Contatore con **do...while**

Funziona anche per contare, come **while** e **for** :

```
int i = 1;

do {
    cout << i << " ";
    i++;
} while (i <= 5);

// Output: 1 2 3 4 5
```

Riepilogo dei tre tipi di ciclo

Ciclo	La condizione viene controllata	Esecuzioni minime
while	Prima del blocco	0 — potrebbe non eseguire mai
for	Prima del blocco	0 — potrebbe non eseguire mai
do...while	Dopo il blocco	1 — esegue almeno una volta

Il `do...while` è meno frequente degli altri due, ma è la scelta più naturale quando sai con certezza che il blocco deve essere eseguito almeno una volta.

Esempio pratico: indovina il numero

```
#include <iostream>
#include <cstdlib>
#include <ctime>
using namespace std;

int main() {
    srand(time(0)); // inizializza il generatore di numeri casuali con
    l'ora attuale
    int numeroDaIndovinare = rand() % 100 + 1; // numero casuale tra 1 e
    100
    int tentativo;
    int contatore = 0;

    cout << "Ho pensato un numero tra 1 e 100. Indovinalo!" << endl;

    do {
        cout << "Tentativo " << (contatore + 1) << ": ";
        cin >> tentativo;
        contatore++;

        if (tentativo < numeroDaIndovinare) {
            cout << "Troppo basso!" << endl;
        } else if (tentativo > numeroDaIndovinare) {
            cout << "Troppo alto!" << endl;
        }

    } while (tentativo != numeroDaIndovinare); // ripeti finché non
    indovina

    cout << "Bravo! Hai indovinato in " << contatore << " tentativi!" <<
    endl;

    return 0;
}
```

Ciclo for

Come usare il ciclo for in C++ per iterazioni controllate.

A cosa serve il ciclo **for** ?

Immagina di dover stampare i numeri da 1 a 100. Scriveresti 100 righe di codice? Certo che no.

Il ciclo **for** ti permette di ripetere un blocco di codice un numero preciso di volte. Scrivi il codice una volta sola, e il programma lo esegue quante volte vuoi.

La struttura del ciclo **for**

```
for (inizio; condizione; aggiornamento) {  
    // codice da ripetere  
}
```

Un esempio concreto:

```
for (int i = 0; i < 5; i++) {  
    cout << i << endl;  
}
```

Output:

```
0  
1  
2  
3  
4
```

Le tre parti del **for**

```
for (int i = 0; i < 5; i++) {  
    //      ^^^^^^^^^  ^^^^^  ^^^  
    //  inizio      cond.   aggiorn.
```

1. **Inizio** (`int i = 0`): viene eseguita una sola volta, all'inizio. Crea il "contatore" del ciclo.
2. **Condizione** (`i < 5`): viene controllata prima di ogni ripetizione. Se è falsa, il ciclo si ferma.
3. **Aggiornamento** (`i++`): viene eseguito alla fine di ogni ripetizione. Di solito aumenta il contatore.

Pensa al ciclo `for` come a un cronometro: parti da zero, vai avanti di uno in uno, ti fermi quando arrivi al limite.

Contare in avanti

```
// Da 1 a 10
for (int i = 1; i <= 10; i++) {
    cout << i << " ";
}
// Output: 1 2 3 4 5 6 7 8 9 10
```

Contare all'indietro

```
// Da 10 a 1 – basta cambiare inizio, condizione e aggiornamento
for (int i = 10; i >= 1; i--) {
    cout << i << " ";
}
// Output: 10 9 8 7 6 5 4 3 2 1
```

Passo diverso da 1

Non devi necessariamente aumentare di uno alla volta:

```
// Solo i numeri pari da 0 a 10
for (int i = 0; i <= 10; i += 2) {
    cout << i << " ";
}
// Output: 0 2 4 6 8 10

// Multipli di 5
for (int i = 5; i <= 25; i += 5) {
    cout << i << " ";
}
// Output: 5 10 15 20 25
```

Cicli annidati

Puoi mettere un ciclo dentro un altro. Utile per lavorare con griglie, tabelle o schemi:

```
// Per ogni riga (i) eseguiamo tutto il ciclo delle colonne (j)
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 3; j++) {
        cout << i * j << "\t"; // \t è il tabulatore (allinea le colonne)
    }
    cout << endl; // vai a capo dopo ogni riga
}
```

Output (tabellina pitagorica 3x3):

```
1  2  3
2  4  6
3  6  9
```

Il for su una collezione (range-based for)

In C++ moderno puoi usare una forma più semplice di `for` per scorrere gli elementi di una lista o di una stringa, senza gestire manualmente l'indice:

```
// Scorre ogni carattere della stringa nome
string nome = "Alice";
for (char c : nome) {
    cout << c << " ";
}
// Output: A l i c e

// Scorre ogni numero dell'array
int numeri[] = {10, 20, 30, 40, 50};
for (int n : numeri) {
    cout << n << " ";
}
// Output: 10 20 30 40 50
```

La sintassi è: `for (tipo elemento : collezione)` . Si legge "per ogni elemento nella collezione".

Usare **auto** per non specificare il tipo

Se non vuoi scrivere il tipo dell'elemento, puoi usare `auto` e lasciare che il compilatore lo capisca da solo:

```
int numeri[] = {1, 2, 3, 4, 5};
for (auto n : numeri) {
    cout << n * 2 << " "; // stampa il doppio di ogni numero
}
// Output: 2 4 6 8 10
```

Esempio pratico: tabella delle potenze

```
#include <iostream>
using namespace std;

int main() {
    // Intestazione della tabella
    cout << "n\tn2\tn3" << endl;
    cout << "----\t----\t----" << endl;

    // Calcoliamo n2, n3 per ogni numero da 1 a 10
    for (int n = 1; n <= 10; n++) {
        cout << n << "\t" << n*n << "\t" << n*n*n << endl;
    }

    return 0;
}
```

Output:

n	n ²	n ³
----	----	----
1	1	1
2	4	8
3	9	27
4	16	64
5	25	125
...		

Ciclo while

Come ripetere istruzioni con il ciclo while in C++.

A cosa serve il ciclo **while** ?

A volte non sai in anticipo quante volte dovrai ripetere qualcosa. Per esempio: continua a chiedere all'utente un numero finché non inserisce uno valido. Oppure: elabora dati finché non hai finito la lista.

Il ciclo `while` ripete un blocco di codice **finché una condizione rimane vera**. Si ferma solo quando la condizione diventa falsa.

La struttura del ciclo `while`

```
while (condizione) {  
    // codice da ripetere  
}
```

Un esempio concreto:

```
int i = 1;  
  
while (i <= 5) {  
    cout << i << endl;  
    i++; // aumentiamo i di 1 ad ogni giro  
}
```

Output:

```
1  
2  
3  
4  
5
```

Come funziona, passo per passo:

1. Controlla la condizione `i <= 5`
2. Se è vera, esegue il blocco
3. Torna al punto 1
4. Quando la condizione è falsa, si ferma

Il contatore: non dimenticarlo!

Di solito si usa una variabile **contatore** per tenere traccia dei giri. Devi:

1. Inizializzarla **prima** del ciclo
2. Aggiornarla **dentro** il ciclo

```
int contatore = 0;

while (contatore < 3) {
    cout << "Esecuzione " << contatore + 1 << endl;
    contatore++; // senza questa riga: ciclo infinito!
}
```

Il ciclo infinito: cosa evitare

Se la condizione non diventa mai falsa, il ciclo gira per sempre. Il programma si blocca:

```
// ATTENZIONE: questo è un ciclo infinito!
int i = 1;
while (i <= 5) {
    cout << i << endl;
    // manca i++ - i rimane sempre 1, la condizione non cambia mai
}
```

Se ti capita, puoi fermare il programma premendo **Ctrl+C** nel terminale.

Quando il **while** è più adatto del **for**

Il **while** eccelle quando non sai in anticipo quante ripetizioni ci saranno. Per esempio, continuare a leggere input finché l'utente non decide di uscire:

```

int numero;
cout << "Inserisci un numero (0 per uscire): ";
cin >> numero;

while (numero != 0) {
    // calcoliamo il quadrato del numero inserito
    cout << "Il quadrato di " << numero << " è " << numero * numero <<
endl;

    cout << "Inserisci un numero (0 per uscire): ";
    cin >> numero;
}

cout << "Arrivederci!" << endl;

```

Ciclo con accumulatore

Un pattern molto comune: sommare o raccogliere valori uno per uno:

```

int somma = 0;    // accumulatore – parte da zero
int numero;
int n = 5;        // quanti numeri vogliamo leggere

cout << "Inserisci " << n << " numeri:" << endl;

int i = 0;
while (i < n) {
    cout << "Numero " << (i + 1) << ": ";
    cin >> numero;
    somma += numero; // aggiungiamo il numero alla somma totale
    i++;
}

cout << "Somma: " << somma << endl;
cout << "Media: " << (double)somma / n << endl;

```

Condizioni complesse

Puoi combinare più condizioni con `&&` e `||` :


```

int tentativi = 0;
int password;
const int PASSWORD_CORRETTA = 1234;

// Continua finché la password è sbagliata E i tentativi sono meno di 3
while (password != PASSWORD_CORRETTA && tentativi < 3) {
    cout << "Inserisci la password: ";
    cin >> password;
    tentativi++;

    if (password != PASSWORD_CORRETTA) {
        cout << "Password errata. Tentativi rimasti: " << (3 - tentativi)
<< endl;
    }
}

if (password == PASSWORD_CORRETTA) {
    cout << "Accesso consentito!" << endl;
} else {
    cout << "Accesso bloccato dopo 3 tentativi." << endl;
}

```

Esempio pratico: calcolatrice interattiva

```
#include <iostream>
using namespace std;

int main() {
    char continua = 's';

    // Continuiamo finché l'utente vuole fare calcoli
    while (continua == 's' || continua == 'S') {
        double a, b;
        char operazione;

        cout << "Inserisci il primo numero: ";
        cin >> a;
        cout << "Operazione (+, -, *, /): ";
        cin >> operazione;
        cout << "Inserisci il secondo numero: ";
        cin >> b;

        if (operazione == '+') {
            cout << "Risultato: " << a + b << endl;
        } else if (operazione == '-') {
            cout << "Risultato: " << a - b << endl;
        } else if (operazione == '*') {
            cout << "Risultato: " << a * b << endl;
        } else if (operazione == '/') {
            if (b != 0) {
                cout << "Risultato: " << a / b << endl;
            } else {
                cout << "Errore: divisione per zero!" << endl;
            }
        }

        cout << "Vuoi continuare? (s/n): ";
        cin >> continua;
    }

    cout << "Ciao!" << endl;
    return 0;
}
```